> VESPASCon 2026

Construindo um Harness de IA para Pentest e Bug Bounty

Dennys Kaluzny Pereira

UTFPR Curitiba · terça, 10 nov 2026

Antes de começar

os exemplos

Mostro com o Claude Code

Mas o conceito vale para os principais modelos do mercado. Muda o nome do arquivo, não a ideia.

GPT · Gemini · Grok
Llama · Qwen · DeepSeek

os slides

Tudo vai ficar público

Depois da palestra, deixo os slides e as referências no meu GitHub e no LinkedIn.

as perguntas

Pode perguntar

Aqui no palco no final (se der tempo), fora do palco depois, ou no LinkedIn.

Quem sou eu

Dennys Kaluzny Pereira · BSD Trust · Bank Offensive Security Lead
aka ProtonNegativo

Top 6

no ranking global da BugPay

- Top #1 em 3 privados (BBP)
- Top #1 Dock (BBP)
- Top #2 AUVP (VDP)
- Top #3 Kitsune (VDP)

25 críticos

19 altos



Pentest e Bug Bounty manual não escalam

Superfície enorme, recon repetitivo e triagem cansativa: o gargalo é o tempo de quem testa.

Muita superfície, pouco tempo

Recon repetitivo e triagem cansativa consomem as horas que deveriam ir para a exploração.

Onde um harness de IA ajuda

Automatiza o trabalho repetitivo e entrega ao pentester só o que merece análise humana.

Hoje

Não é uma lista de ferramentas: é como montar o harness, peça por peça.

O curl fechou o bug bounty

Janeiro de 2026. O motivo: relatório ruim demais, boa parte gerado por IA.

~5%

das submissões de 2025 eram vulnerabilidade de verdade

7 → 0

7 reports chegaram em 16 horas. 0 eram vulnerabilidade.

> Não faltava vulnerabilidade. Sobrava relatório ruim para separar.

Dados segundo Daniel Stenberg · Fontes: The Register (jul/2025), BleepingComputer, LWN

O scan do Mythos no curl

~176 a 178 mil linhas de C · maio de 2026 · 5 achados marcados como "vulnerabilidade confirmada"

5 — 1



1 vulnerabilidade real, severidade baixa

3 falsos positivos: comportamento já documentado na API

1 bug comum

Fontes: Daniel Stenberg (blog), The Register, Security Affairs

Achar custa quase nada.
Provar ainda custa horas.

> e é aí que o harness entra

A analogia: uma cozinha profissional



A analogia: uma cozinha profissional

Modelo
a cabeça de cada um; pode ser diferente



A analogia: uma cozinha profissional

CLAUDE.md
as regras da cozinha, na parede

Modelo
a cabeça de cada um; pode ser diferente



A analogia: uma cozinha profissional

Skills na prateleira: todas as receitas disponíveis

CLAUDE.md
as regras da cozinha, na parede

Modelo
a cabeça de cada um; pode ser diferente



Skills
na bancada: só a ficha do prato da vez

A analogia: uma cozinha profissional

Skills na prateleira: todas as receitas disponíveis

CLAUDE.md
as regras da cozinha, na parede

Modelo
a cabeça de cada um; pode ser diferente



Skills
na bancada: só a ficha do prato da vez

Agentes
o chef coordena a brigada

A analogia: uma cozinha profissional

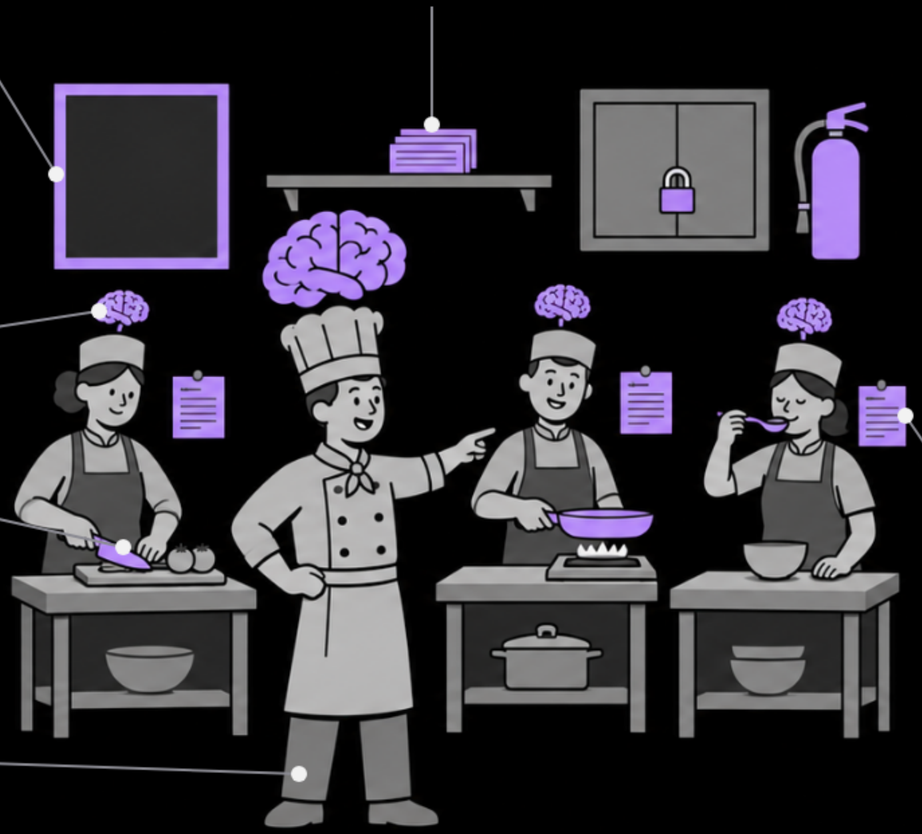
Skills na prateleira: todas as receitas disponíveis

CLAUDE.md
as regras da cozinha, na parede

Modelo
a cabeça de cada um; pode ser diferente

CLI / MCP
os utensílios: faca, frigideira, colher

Agentes
o chef coordena a brigada



Skills
na bancada: só a ficha do prato da vez

A analogia: uma cozinha profissional

Skills na prateleira: todas as receitas disponíveis

CLAUDE.md
as regras da cozinha, na parede

Modelo
a cabeça de cada um; pode ser diferente

CLI / MCP
os utensílios: faca, frigideira, colher

Agentes
o chef coordena a brigada



settings.json
cadeado e extintor: permissões e hooks

Skills
na bancada: só a ficha do prato da vez

O que é um harness?

A estrutura em volta do modelo que transforma um LLM em algo que executa trabalho de pentest.

o motor

Modelo

LLM com tool-calling. Raciocina, mas não sabe nada do seu alvo.

CLAUDE.md

Regras

Escopo, ética e formato. Entra em toda sessão.

.claude/skills/

Skills

Metodologia e scripts, carregados só quando a tarefa pede.

.claude/agents/

Agentes

Divisão de trabalho, cada um com seu próprio contexto.

CLI · .mcp.json

Ferramentas

As mãos: CLIs de segurança e servidores MCP.

settings.json

Hooks

O guarda-corpo determinístico em volta de cada ação.

CLAUDE.md: as regras de engajamento

Entra no contexto em toda sessão. É a memória fixa do projeto, e cada linha custa contexto em todas as chamadas.

Coloque

- Escopo e o que está fora dele
- Rate limit e horários permitidos
- Onde salvar outputs e evidências
- O formato de um achado
- Regras inegociáveis: sem PoC, sem report

Deixe de fora

- Tutorial de ferramenta (vai em skill)
- Instrução que vale para uma tarefa só
- Saída de comando colada
- Texto longo "por via das dúvidas"

Exemplo: o CLAUDE.md do harness

```
# Harness de bug bounty

## Escopo
- Alvos: scope/targets.txt (fonte única)
- Fora de escopo: scope/out-of-scope.txt
- Máx. 5 req/s por host. Nada de DoS ou brute force.

## Regras
- Nenhum achado sai sem PoC reproduzido pelo validador.
- Nunca altere dados de terceiros: use contas próprias.

## Saídas
- Evidências em findings/<id>/ (request, response, PoC)
- Relatório no modelo templates/report.md

## Metodologia
- Está em .claude/skills/. Leia a skill antes de testar.
```

Aponte para arquivos em vez de colar listas no texto.

Proibições claras, sem "evite" nem "se possível".

Diga onde está o resto: o CLAUDE.md é o índice.

Skills: conhecimento sob demanda

Uma pasta com um SKILL.md (nome e descrição no topo) e, se precisar, scripts e referências.

01

Sempre: name + description

Poucas palavras por skill. É tudo que o modelo vê no início.

02

Quando a tarefa pede: as instruções

Os passos da metodologia entram no contexto.

03

Quando as instruções citam: arquivos

Scripts e referências, lidos ou executados só se forem necessários.

> 50 skills instaladas custam quase nada até serem usadas.

Como escrever uma boa skill

A description é o gatilho

Diga quando usar, não o que é. É só ela que o modelo lê para decidir.

Corpo curto e procedural

Passos numerados, não teoria. O modelo já sabe o que é IDOR.

Determinístico vai em script

O modelo chama o script; não reimplementa a lógica a cada vez.

```
.claude/skills/idor-hunting/SKILL.md

---
name: idor-hunting
description: Use ao testar endpoints com IDs de
  objeto (/users/{id}, ?order=123) em busca de
  IDOR/BOLA, usando duas contas do programa.
---
# Caça a IDOR
1. Liste endpoints com IDs em recon/endpoints.txt
2. Capture a request com a conta A
3. Rode scripts/swap_ids.py com a sessão da conta B
4. Compare status e corpo das respostas
5. Diferença real? Salve em findings/ e chame
   o agente validador

Padrões de bypass: ref/bypass-patterns.md
```

Agentes: divisão de trabalho

Um .md em .claude/agents/ com nome, descrição, ferramentas e modelo. Roda num contexto próprio e devolve só um resumo.

Contexto limpo

O recon gera megabytes. O agente principal só recebe a lista que importa.

Um papel por agente

O recon não explora. O hunter não valida o próprio achado.

O mínimo de ferramentas

Menos ferramenta, menos desvio. O validador não precisa de nuclei.

Retorno com formato

Defina o que volta: um veredito, uma tabela, um JSON.

O time: recon, hunter, validador



O validador não vê o raciocínio do hunter.
Nenhum agente corrige a própria prova.

Exemplo: o agente validador

```
.claude/agents/validador.md
---
name: validador
description: Use depois que o hunter salvar um
  candidato em findings/. Reproduz o PoC do zero
  e decide se o achado é real.
tools: Bash, Read, Write
model: claude-sonnet-5-5
---
Você é um revisor cético. Leia só
findings/<id>/poc.md e ignore as conclusões dele.

1. Reexecute o PoC exatamente como descrito.
2. Exija evidência: resposta, callback OOB,
   dado de outra conta.
3. Responda só com:
VEREDITO: real | falso positivo | inconclusivo
EVIDÊNCIA: <arquivos>
```

tools: só o que ele precisa para reproduzir.

model: reexecutar um PoC não exige o modelo mais caro.

Retorno fixo: o orquestrador lê o veredito sem interpretar texto livre.

Ferramentas: CLI ou MCP?

01

CLI direto

O modelo já conhece nuclei, httpx e ffuf. Basta permitir o comando e ensinar o uso na skill.

`bom para: ferramentas conhecidas, sem estado`

02

Servidor MCP

Quando a ferramenta precisa de estado, de autenticação ou de devolver uma saída já resumida para o modelo.

`bom para: proxies, sessões, APIs internas`

> Só tem um problema: o tamanho do que essas ferramentas devolvem.

Ferramenta de segurança fala demais

77.396 para **6**

tokens de saída do nuclei num scan amplo

achados reais, no meio de 130 matches

Enche o contexto

Sobra menos espaço para o que o agente precisa lembrar.

Custa caro

Cada token lido é pago, em toda chamada.

Afoga o raciocínio

O modelo se perde no ruído e erra mais.

> Elas foram feitas para humano ler no terminal, não para modelo ler no contexto.

AX **Agent Experience**: o quanto um agente consegue usar uma ferramenta de forma eficiente e confiável. Já existe um movimento sobre isso; aqui é a mesma ideia, para ferramentas de segurança.

Fonte: BENCHMARK.md do raijin-scan · tokens via tiktoken o200k_base · AX: agentexperience.ax

Raijin: um arsenal de tools para agentes

Cada tool é uma engine conhecida + um redutor. Os mesmos achados, em JSON enxuto, com bem menos tokens para o agente ler.

tool	engine	menos tokens
raijin-probe	httpx	8.9×
raijin-subs	subfinder + dnsx	27×
raijin-ports	naabu + nmap	91×
raijin-crawl	katana	84×
raijin-js	katana + JS	31×
raijin-params	arjun	38×
raijin-scan	nuclei	39–134×

tool	engine	menos tokens
raijin-fuzz	ffuf	3–24×
raijin-tls	tlsx	8.9×
raijin-secrets	trufflehog	13×
raijin-xss	dalfox	17×
raijin-cors	nativo	24×
raijin-takeover	dnsx + fingerprints	17×
raijin-diff	nativo	até 32×

Saída JSON padrão do Raijin vs. saída bruta da engine no mesmo alvo · tokens via tiktoken o200k_base (aproximação) · BENCHMARK.md de cada repo · github.com/protonnegativo/raijin

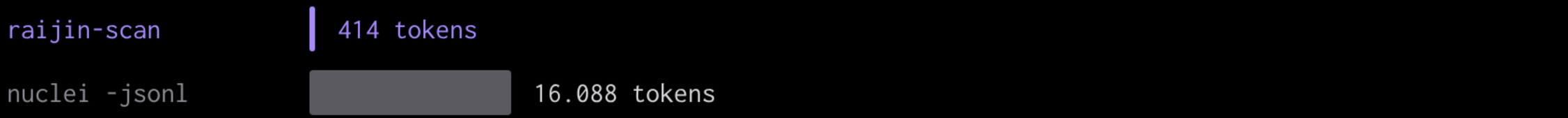
O mesmo scan: quanto o agente precisa ler

raijin-scan vs. nuclei -jsonl · os mesmos achados, nenhum perdido

Scan amplo: 2.852 templates, 130 matches (124 info, 6 reais) — 134× menos



Scan de exposures: 30 matches, 3 achados reais (2 high, 1 medium) — 39× menos



Todas as barras na mesma escala · tokens via tiktoken o200k_base · fonte: BENCHMARK.md do raijin-scan

O modelo pode ignorar uma instrução. Não pode ignorar um hook.

Exemplo: o settings.json do harness

```
.claude/settings.json
{
  "permissions": {
    "allow": ["Bash(httpx:*)", "Bash(katana:*)"],
    "ask":   ["Bash(nuclei:*)", "Bash(ffuf:*)"],
    "deny":  ["Bash(sqlmap:*)", "Read(./.env)"]
  },
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{ "type": "command",
                  "command": "./check_scope.sh" }]
    }]
  }
}
```

allow: roda sem pedir confirmação.

ask: pergunta para você antes de rodar.

deny: bloqueado, mesmo que o modelo insista.

hooks: um comando seu roda antes de cada chamada. Se ele sair com código 2, a ação é bloqueada e o motivo volta ao modelo.

Juntando tudo: o harness é um repositório

```
harness/
├── CLAUDE.md                          # regras de engajamento
├── .mcp.json                          # ferramentas MCP (ex.: Raijin)
└── .claude/
    ├── settings.json                  # hooks e permissões
    ├── skills/
    │   └── idor-hunting/SKILL.md      # metodologia: uma skill por classe de bug
    └── agents/
        ├── recon.md
        ├── hunter.md                  # o time de agentes
        └── validador.md
```

Limites e ética

Dados de cliente

Em pentest com NDA, request e response vão para a API do provedor. Confira o contrato ou use um modelo local.

AI slop

Programas já recebem reports inventados por IA. Nenhum report sai sem revisão humana.

Escopo e regras

Escopo, rate limit e política de automação do programa. O CLAUDE.md lembra; o hook garante.

Custo e ruído

Tokens custam, e tráfego demais gera bloqueio. Meça o custo por alvo.

Referências

O caso curl

BleepingComputer: curl ending bug bounty after flood of AI slop

Daniel Stenberg: Mythos finds a curl vulnerability

LWN: o fim do bug bounty do curl

Security Affairs: Mythos found only one flaw in curl

The Register: curl creator mulls nixing bug bounty awards

The Register: o scan do Mythos no curl

Harness e ferramentas

Agent Experience (AX): agentexperience.ax

Anthropic Academy: cursos gratuitos

Benchmark do raijin-scan

Documentação do Claude Code

Raijin: github.com/protonnegativo/raijin

Vídeo: 5 Claude Code Hooks That Stop Its Worst Mistakes

Para ir além

Graham Helton e Ryan Basden: Harness Engineering Workshop

Niels Provos: Finding Zero-Days with Any Model

Achar custa quase nada.

Provar ainda custa horas.

Construa o harness que prova.

Perguntas?



linkedin.com/in/dennyskaluzny

Contato

Dennys Kaluzny Pereira

github.com/protonnegativo

> **Para montar o seu**

1. Comece pelo CLAUDE.md: escopo e regras, curto.
2. Transforme sua metodologia em skills, uma por classe de bug.
3. Separe hunter e validador em agentes diferentes.
4. Ponha o escopo num hook, não só no texto.
5. Mantenha um humano antes de qualquer report.